

Retrieval-Augmented CV Enhancer

Liubomyr Bobrovych, bobroliu@fit.cvut.cz

1 Introduction

The goal of this project was to design and implement a web application that rewrites a candidate's CV to better match a target job advertisement. The user provides as input either a DOCX or PDF file that is preprocessed and parsed. The system combines a **TF-IDF** and a **sentence-transformer** retrieval system, merged by **Reciprocal Rank Fusion (RRF)**. To support the editing of the CV, **LLM** calls are implemented to finally output a PDF-formatted enhanced CV.

2 Solution Methods

2.1 Document and Job Ad Preprocessing

Given a DOCX or PDF on the input, format-specific parsing strategies are used. For PDF, text is extracted via **pypdf** with pages joined by blank lines. For DOCX, the OXML body is walked in document order using **python-docx**, reconstructing both paragraphs and tables to preserve the original content sequence.

Raw job ads contain 30–60% boilerplate (benefits, culture statements, application instructions) that would pollute retrieval. A nearest-centroid classifier in sentence-transformer embedding space filters this: hand-written anchor sentences define “role content” and “benefits content” centroids, and each block of the ad is assigned to whichever centroid it is closer to. Only role-relevant blocks are kept.

2.2 Chunking

CVs follow a semi-structured format: they typically contain a header with contact information, a summary or objective section, a list of work experiences with dates, and trailing sections for education, skills, and languages. The chunker exploits this structure in **role-aware mode**: a regex detects date patterns at line endings (e.g. “2020 – Present”, “06/2025”) to identify role boundaries, and each role becomes one chunk. Content before the first role (contact info, summary) and after the last (education, skills) is chunked separately by blank lines. If fewer than two role headers are found, a **fallback mode** splits on blank lines and merges short blocks up to approximately 400 characters.

2.3 Retrievers: TF-IDF and Embeddings

The retrieval pipeline uses two complementary methods.

TF-IDF is implemented from scratch (without sklearn's `TfidfVectorizer`) following the course lecture formulation. The weighting scheme is:

$$\text{TF}(t, d) = \frac{f(t, d)}{\max_{t'} f(t', d)}, \quad \text{IDF}(t) = \log_2 \frac{N}{\text{df}(t)}, \quad w(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \quad (1)$$

The pipeline proceeds as follows:

1. **Tokenization** – lowercase, regex word extraction (`\w+`), English stop-word removal.
2. **N-grams** – bigrams are formed from adjacent unigrams to capture compound terms like “machine learning” or “data science”.
3. **TF-IDF weighting** – max-norm term frequency and $\log_2$ inverse document frequency as in Equation (1).
4. **Inverted index** – stored as a `scipy.sparse.csr_matrix`.
5. **Cosine similarity** – all vectors are L2-normalized, so ranking reduces to a dot product:

$$\cos(q, d) = \hat{q} \cdot \hat{d}, \quad \text{where } \hat{v} = \frac{v}{\|v\|_2} \quad (2)$$

TF-IDF is well suited for the CV-enhancement task because job ads use specific terminology (“Python”, “Kubernetes”, “CI/CD”) and exact term matching is essential. Bigrams further improve precision: “machine learning” as a single token is far more discriminating than the two unigrams separately. However, TF-IDF misses paraphrases: “built APIs” and “developed web services” share zero surface terms. This is why a **dense retriever** (all-MiniLM-L6-v2 sentence embeddings) is used alongside it: it captures synonyms and semantic similarity that bag-of-words models cannot.

2.4 Reciprocal Rank Fusion

The two retrievers produce independent rankings. These are combined using RRF:

$$\text{score}_{\text{RRF}}(d) = \frac{1}{k + \text{rank}_{\text{tfidf}}(d)} + \frac{1}{k + \text{rank}_{\text{dense}}(d)}, \quad k = 60 \quad (3)$$

RRF aggregates rank positions rather than raw scores. TF-IDF cosines and dense cosines live on incomparable scales, so a weighted sum would be dominated by whichever method produces larger numbers. By using ranks only, RRF is scale-invariant and rewards consensus between the two methods.

2.5 LLM Rewriting

The user selects which chunks to enhance via a selection UI. The selected chunks, together with the full CV and the filtered job ad, are sent to Gemini 2.5 Flash in a single API call. The prompt instructs the model to rewrite only the selected chunks, preserve everything else, and never fabricate skills or experience. The response is structured via a Pydantic schema enforced through LangChain’s `.with_structured_output()`. This architecture separates retrieval (which chunks matter) from generation (how to rewrite them), giving the user explicit control and keeping the LLM’s role narrow.

3 Implementation

The application is written in **Python 3.11** using **Streamlit** as the web framework, chosen for its simplicity in building data-oriented UIs with minimal frontend code. The environment is managed with **conda** (`environment.yml`).

Key libraries: **pypdf** and **python-docx** for document parsing; **scipy** (sparse matrices) and **numpy** for the TF-IDF matrix and cosine computation; **sentence-transformers** for dense embeddings; **langchain-google-genai** for the Gemini API call; **pydantic** for response schema validation; **reportlab** for PDF rendering of the enhanced CV.

Gemini 2.5 Flash was chosen as the LLM because it offers a free-tier API with sufficient quality for structured rewriting tasks. The application is deployed on Streamlit Community Cloud with the API key stored as a Streamlit Cloud secret.

4 Showcase of the Application

The following screenshots show the three main stages of the application workflow.

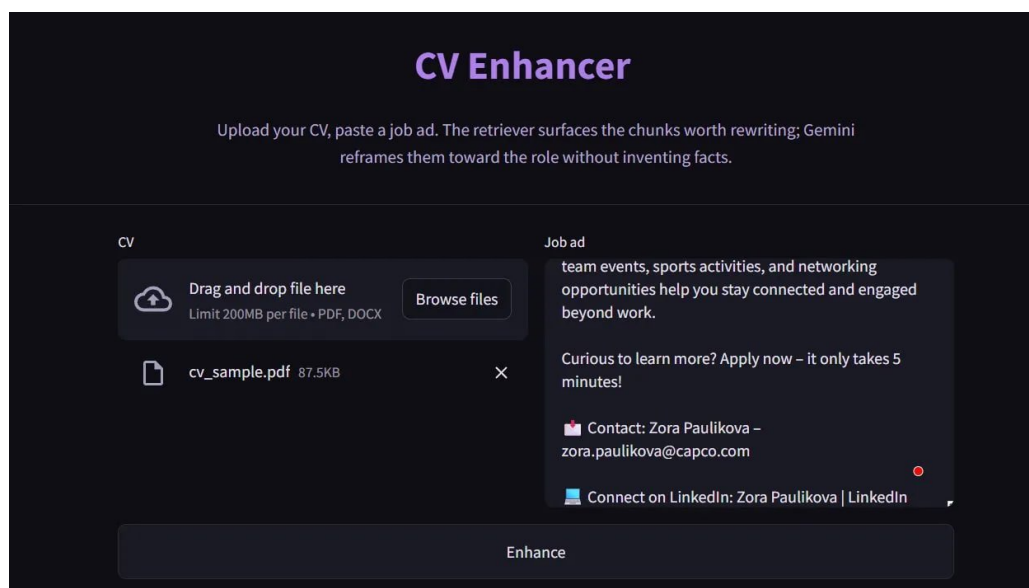


Figure 1: The user uploads a CV (PDF or DOCX) and pastes a job advertisement.

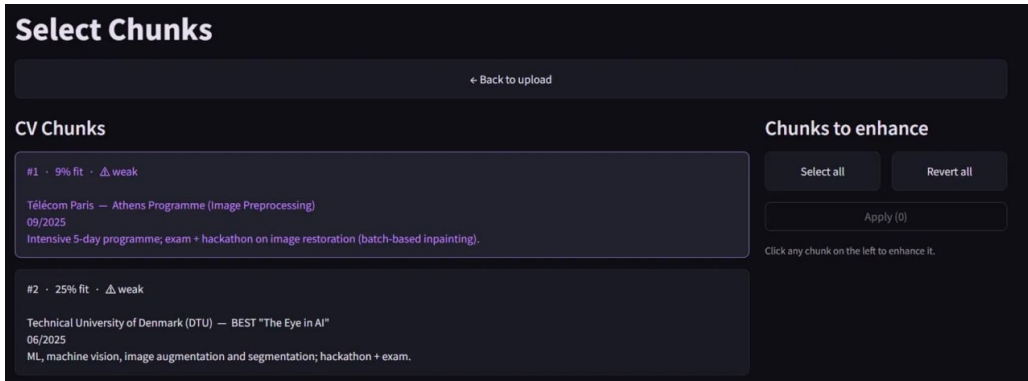


Figure 2: Chunks ranked by relevance to the job ad. Each chunk shows its fit score and strength indicator. The user selects which chunks to enhance.

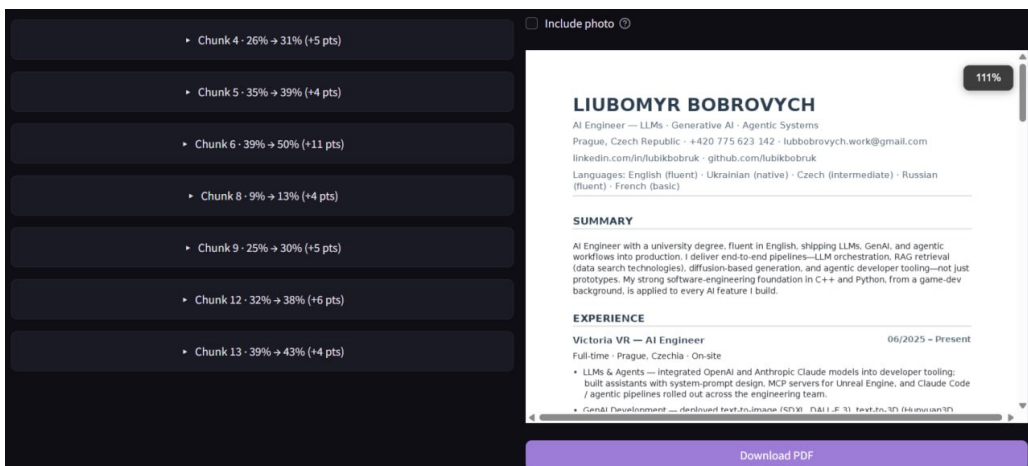


Figure 3: Enhanced CV with per-chunk rewrites, similarity score improvements, and a Download PDF button.

5 Experiments

A stand-alone notebook (`notebooks/ranking_demo.ipynb`) runs all three rankers side-by-side on a synthetic corpus of 8 CV chunks against a backend-engineer job ad. The corpus includes strong fits, paraphrase-heavy matches, and off-topic chunks to make retriever differences visible.

TF-IDF:		
#1	chunk 5	score=0.4328 Skills: Python, Go, PostgreSQL, Redis, Kafka, Docker, Kubernetes,...
#2	chunk 1	score=0.3019 Acme Corp — Senior Backend Engineer 06/2023 – Present. Designe...
#3	chunk 0	score=0.1133 Jane Doe — Backend Software Engineer. Five years building REST an...
Dense (MiniLM):		
#1	chunk 1	score=0.6356 Acme Corp — Senior Backend Engineer 06/2023 – Present. Designe...
#2	chunk 5	score=0.6337 Skills: Python, Go, PostgreSQL, Redis, Kafka, Docker, Kubernetes,...
#3	chunk 0	score=0.5224 Jane Doe — Backend Software Engineer. Five years building REST an...
RRF (fused):		
#1	chunk 5	score=0.0325 Skills: Python, Go, PostgreSQL, Redis, Kafka, Docker, Kubernetes,...
#2	chunk 1	score=0.0325 Acme Corp — Senior Backend Engineer 06/2023 – Present. Designe...
#3	chunk 0	score=0.0317 Jane Doe — Backend Software Engineer. Five years building REST an...

Figure 4: Top-3 results from TF-IDF, Dense (MiniLM), and RRF on the synthetic corpus.

The results confirm the expected behavior:

- **TF-IDF** ranks the Skills chunk highest (score 0.43) due to exact keyword overlap with the job ad.
- **Dense retriever** ranks the Acme Corp role highest (score 0.64) because it captures semantic similarity beyond surface terms.
- **RRF** surfaces both as tied at #1 (score 0.0325), rewarding consensus between the two methods.
- Off-topic chunks receive zero from TF-IDF but non-zero (small) scores from the dense method, illustrating why rank-based fusion is more robust than score averaging.

Speed is not a concern for the retrieval stage, since the corpus consists of chunks from a single document, guaranteeing a relatively small number of candidates. The main performance bottleneck is the LLM call: the choice of model and the number of chunks selected for rewriting directly determine the overall processing time.

6 Discussion

Strengths

The project delivers a working end-to-end pipeline deployed publicly. The TF-IDF retriever is implemented from course theory without relying on sklearn’s vectorizer. The dense retriever and RRF fusion extend this with modern techniques while keeping the graded core independent. The chunk selection UI gives users explicit control over what gets rewritten, and the anti-fabrication constraint in the LLM prompt prevents the model from inventing skills.

Limitations

The main limitation is **document formatting and consistency**. The PDF parser loses bullet glyphs, occasionally reorders content on multi-column templates, and produces empty text on scanned PDFs. A repair pass partially compensates, but edge cases remain. Replacing the parser with pdfplumber or pymupdf and adding OCR fallback would improve coverage.

The job-ad filter uses hand-written prototypes rather than a trained classifier. No quantitative evaluation (Precision@ k , MRR) was conducted due to the absence of a labeled dataset; the evaluation skeleton is scaffolded and ready for future use. The system is English-only; a multilingual embedding model would extend support to other languages.

7 Conclusion

The project implements a retrieval-augmented CV enhancement system covering the core course topics: hand-rolled TF-IDF, dense semantic retrieval, and rank aggregation via RRF. The LLM layer consumes retrieval output as context without replacing the search model. The main practical challenge was data extraction: parsing real-world PDFs and DOCX files into clean chunks. The application is deployed at <https://cv-enhancer.streamlit.app/> and demonstrates the full pipeline from upload to enhanced PDF download.